Lab 08 — Configuratie en secrets

Theorie — hoe één enkele image dient in ontwikkeling, acceptatie en productie zonder herbouwd te worden, en waarom een wachtwoord dat in een image, een commandoregel of een log belandt, daar nooit meer uit komt.

Doelstellingen

- Begrijpen waarom configuratie **buiten de image** moet zitten, en wat het *12-factor*-principe zegt.
- `ENV`, `-e` en `--env-file` beheersen, hun volgorde van voorrang en de conventie `_FILE`.
- Omgevingsvariabelen koppelen aan de configuratie van Spring Boot (*relaxed binding*, profielen).
- Weten **waar een secret zichtbaar is** (image, `inspect`, `exec`, logs, `ps`) en dus waar het niet thuishoort.
- `podman secret` gebruiken bij het uitvoeren en `--mount=type=secret` bij het bouwen.

---

1. Eén image, drie omgevingen

In lab 05 bouwde je `api:1.0`. Die image moet draaien op je werkstation tegen een lokale database, op de acceptatieserver tegen een testdatabase, en in productie tegen de echte database — met andere URL's, andere wachtwoorden en andere logniveaus. Had elke omgeving haar eigen image, dan zou je in acceptatie een image testen **die niet** dezelfde is als die in productie gaat. De hele discipline van het uitrollen steunt op het omgekeerde: **één onveranderlijke image, die van omgeving naar omgeving gepromoveerd wordt, en configuratie die van buitenaf ingespoten wordt**.

> **→ GESCHIEDENIS**
> In 2011 publiceren de ingenieurs van Heroku *The Twelve-Factor App*, twaalf regels voor applicaties die in de cloud uitrolbaar zijn. Factor III, *Config*, wordt het vaakst aangehaald: "sla de configuratie op in de omgeving", omdat omgevingsvariabelen onafhankelijk zijn van de taal, het systeem en het configuratiebestand, en omdat je ze niet per ongeluk in Git commit. Docker (2013) maakte er zijn standaardpraktijk van: `-e`.

Wat verschilt tussen omgevingen is **configuratie**: database-URL, inloggegevens, poorten, logniveaus, feature flags. Wat niet verschilt is **code**, en die leeft in de image. En binnen de configuratie is alles wat toegang geeft tot iets een **secret**: wachtwoord, API-sleutel, privécertificaat. De drie behandel je niet op dezelfde manier.

2. De mechanismen

Mechanisme	Waar	Bereik	Gebruik
`ENV` in de Dockerfile	in de image	alle containers van de image	een redelijke **standaardwaarde**, nooit een secret
`-e SLEUTEL=waarde`	`podman run`	deze container	een of twee waarden overschrijven
`--env-file api.env`	`podman run`	deze container	een set waarden per omgeving, bestand buiten Git
gemount bestand (`-v ./application.yaml:/config/…:ro`)	`podman run`	deze container	gestructureerde, lange, geversioneerde configuratie
`--secret` (`/run/secrets/…`)	`podman run`	deze container	wachtwoorden, sleutels, certificaten
commandoargument	`podman run … --wachtwoord=x`	deze container	**nooit** voor een secret (zichtbaar in `ps` , `logs` , `inspect`)

De voorrang ligt vast: **`-e` overschrijft `--env-file`, dat `ENV` van de image overschrijft**, ongeacht de volgorde van de opties op de regel. En `-e SLEUTEL` zonder waarde geeft de variabele **van de shell van de host** door: handig in CI, een valkuil bij het nalezen.

→ LINUX
Omgevingsvariabelen zijn een blok tekst `SLEUTEL=waarde` dat elk proces bij zijn aanmaak van zijn ouder krijgt, en dat elk kind erft. Ze zijn versleuteld noch beschermd: `/proc/<pid>/environ` toont ze aan elk proces van dezelfde gebruiker, en `podman exec container env` drukt ze af. Een programma kan ze niet "vergeten" eens het gestart is — daarom is een secret in een variabele **leesbaar**, maar ook daarom is het zo eenvoudig.

→ SPRING BOOT
Spring Boot leest **al** zijn properties uit de omgeving dankzij *relaxed binding*: `spring.datasource.url` wordt `SPRING_DATASOURCE_URL` , `server.port` wordt `SERVER_PORT` , en `SPRING_PROFILES_ACTIVE=prod` activeert het profiel `prod` (dus `application-prod.yaml`). Een bestand gemount op `/config/application.yaml` wordt automatisch gelezen, en `SPRING_CONFIG_ADDITIONAL_LOCATION` wijst naar andere. De volgorde: omgevingsvariabele > extern bestand > bestand in de JAR. Precies dezelfde logica als die van de containerengine, één verdieping hoger.

→ JAVA
Ook de JVM leest een variabele: `JAVA_TOOL_OPTIONS` . De inhoud ervan wordt aan de opstartopties toegevoegd, en de JVM meldt dat met `Picked up JAVA_TOOL_OPTIONS: …` op de foutuitvoer. Zo stel je het geheugen in (`-Xmx` , `-XX:MaxRAMPercentage`) of koppel je een agent zonder de `ENTRYPOINT` van de image aan te raken — lab 10 gebruikt dit voor de geheugenlimieten.

3. Waar een secret zichtbaar is

Maak de inventaris voor je een mechanisme kiest. Een secret dat…

- **in de image** zit (`ENV` , `COPY .env` , `ARG` — lab 04), is leesbaar via `podman history` , `podman image inspect` , `podman run --rm image cat /app/.env` , en voor iedereen die de image uit de

registry haalt. **Nooit.**

- **als commandoargument** meegegeven wordt (`--db-password=x`), verschijnt in `podman inspect` (`Config.Cmd`), `podman top`, de `ps` van de host, en vaak in de logs van de applicatie die haar argumenten afdrukt. **Nooit.**
- **als omgevingsvariabele** doorgegeven wordt (`-e`, `--env-file`), verschijnt in `podman inspect` (`Config.Env`), `podman exec env`, `/proc/1/environ`, en in elk foutrapport of elke *dump* die de omgeving bevat. **Aanvaardbaar** op een werkstation of in CI, **ontoereikend** in productie.
- **in een alleen-lezen gemount bestand** of een secret van de engine zit, verschijnt noch in `inspect`, noch in de omgeving; alleen het proces dat het bestand leest, ziet het. Dat is het doel.

VALKUIL
Bijna alle officiële images bieden de conventie `_FILE`: `POSTGRES_PASSWORD_FILE=/run/secrets/db_password` laat het wachtwoord uit een bestand lezen in plaats van uit `POSTGRES_PASSWORD`. De omgeving bevat dan alleen nog een **pad**. Ga na of je eigen applicatie dat ook kan: Spring Boot doet het met `spring.config.import=optional:configtree:/run/secrets/`, dat elk bestand in de map omzet in een property.

4. De secrets van de engine

`podman secret create db_password -` leest een secret van de standaardinvoer en bergt het op in de opslag van je gebruiker; `podman secret ls` somt het op, `podman secret inspect` toont de metadata zonder de waarde, `podman secret rm` verwijdert het. Bij `run` mount `--secret db_password` het in de container onder `/run/secrets/db_password`, alleen-lezen, en **er verschijnt niets in `podman inspect`**. De variant `--secret db_password,type=env,target=POSTGRES_PASSWORD` spuit het in als omgevingsvariabele voor applicaties die geen bestand lezen — zichtbaar via `exec env`, maar nog altijd afwezig in `inspect`.

PODMAN
`podman secret` werkt zonder enige orchestrator. Bij Docker vereist `docker secret` de Swarm-modus; buiten Swarm val je terug op de `secrets:` van Compose, gewone gemounte bestanden — die `podman compose` ook begrijpt. Qua opslag bewaart de standaarddriver van Podman (`file`) de waarde **gecodeerd, niet versleuteld**, in `~/.local/share/containers/storage/secrets/`, en `podman secret inspect --showsecret` leest ze terug: het is een opbergplaats, geen kluis. Quadlet (lab 10) kan er een secret naar verwijzen, en Kubernetes heeft zijn object `Secret` (lab 14), met dezelfde beperking.

→ BEVEILIGING
In het bedrijf leven secrets in een **secretsbeheerder** (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault): versleuteld in rust, toegang gelogd, automatische **rotatie**. De container krijgt ofwel een bestand dat bij het starten ingespoten wordt, ofwel een kortlevend token om het secret zelf op te halen. Het doel is niet "dat niemand het kan lezen", maar "weten wie het gelezen heeft, en het in één minuut kunnen veranderen".

5. Secrets bij het bouwen

Een `npm install` of `mvn package` heeft soms een token nodig voor een privérepository, **alleen tijdens** het bouwen. Lab 04 toonde dat `ARG` in `podman history` belandt. Het juiste antwoord is de secret mount: `RUN --mount=type=secret,id=token,required=true cat /run/secrets/token` maakt

het bestand beschikbaar voor **die ene instructie**, en `podman build --secret id=token,src=token.txt` levert het aan. Het wordt in geen enkele laag geschreven, `history` toont alleen de regel `--mount=…`, en `/run/secrets` bestaat niet meer in de image. Zonder `--secret` laat `required=true` de build falen: `secret required but no secret with id "token" found`.

Let op de cache: een `RUN`-instructie die al in de cache zit, **wordt niet opnieuw uitgevoerd** en vraagt het secret dus niet opnieuw. Een verlopen token merk je pas bij de volgende `--no-cache`.

6. Het bestand `.env`

Een bestand `api.env` (of `.env`) per omgeving is de meest verspreide vorm: één regel `SLEUTEL=waarde` per variabele, `#` voor commentaar, geen aanhalingstekens en geen spaties rond het `=`. Drie regels: het staat in `.gitignore` **en** in `.dockerignore` (lab 04); een `api.env.example` zonder waarden wordt in zijn plaats gecommit; en het bevat geen productiesecrets — daarvoor gebruik je een secret of een gemount bestand.

7. In het bedrijf

- **Eén image, gepromoveerd**: de CI bouwt `api:1.4.2` één keer, pusht ze, en dezelfde digest wordt uitgerold in acceptatie en daarna in productie, met andere variabelen.
- **Spring-profielen via variabele**: `SPRING_PROFILES_ACTIVE=prod` in de uitrol, nooit in de Dockerfile; `application-prod.yaml` bevat geen wachtwoord maar `${DB_PASSWORD}`, opgelost uit de omgeving of een secretsbestand.
- **De secrets**: bestanden ingespoten door de orchestrator of de secretsbeheerder, `_FILE` of `configtree`, geplande rotatie. Een secret dat in een log, een ticket of een `inspect` opduikt, geldt als **gecompromitteerd** en wordt vervangen.
- **De review**: een `ENV *_PASSWORD` of een `COPY .env` in een Dockerfile wordt bij de codereview geweigerd, automatisch als het kan (*linters* zoals Hadolint, scanners zoals Trivy of Gitleaks).

Onthouden

- Eén onveranderlijke image, ingespoten configuratie: `ENV` voor de standaardwaarden, `-e` / `--env-file` voor de omgeving, bestanden of secrets voor wat gevoelig is.
- Vaste voorrang: `-e` > `--env-file` > `ENV` van de image.
- Spring Boot leest alles uit de omgeving (`SPRING_DATASOURCE_URL`, `SPRING_PROFILES_ACTIVE`); de JVM leest `JAVA_TOOL_OPTIONS`.
- Een secret als argument is zichtbaar in `ps`, `inspect` en de logs; als variabele in `inspect` en `exec env`; als gemount bestand of `--secret` nergens, behalve voor het proces.
- `_FILE` of `configtree`: de omgeving draagt alleen een pad.
- `podman secret` bergt op zonder te versleutelen; de echte kluis is een secretsbeheerder met rotatie.
- Bij het bouwen: `--mount=type=secret`, nooit `ARG`; en de cache vraagt een secret niet opnieuw.

Woordenschat

12-factor: twaalf regels voor uitrolbare applicaties, waaronder "config in de omgeving". — **configuratie / secret**: wat per omgeving verschilt / wat toegang geeft. — **`ENV`**: standaardvariabele vastgelegd in de image. — **`--env-file`**: bestand `SLEUTEL=waarde` geladen bij

`run`. — **relaxed binding**: vertaling door Spring Boot van properties naar variabelen. — **Spring-profiel**: configuratieset geactiveerd door `SPRING_PROFILES_ACTIVE`. — **conventie `_FILE`**: een waarde lezen uit een bestand waarvan het pad in een variabele staat. — **`podman secret`**: secret beheerd door de engine, gemount in `/run/secrets/`. — **build-secret**: bestand dat beschikbaar is tijdens één enkele `RUN`-instructie. — **secretsbeheerder**: versleutelde, gelogde dienst met rotatie.